

What is Concurrency in OS?

Concurrency in an operating system refers to the ability to execute multiple processes or threads simultaneously, improving resource utilization and system efficiency. It allows several tasks to be in progress at the same time, either by running on separate processors or through context switching on a single processor. Concurrency is essential in modern OS design to handle multitasking, increase system responsiveness, and optimize performance for users and applications.

Need for Concurrency

1. **Resource Utilization:** Concurrency ensures better utilization of CPU, memory, and I/O devices by keeping them busy.
2. **Responsiveness:** It allows the system to remain responsive even when multiple tasks are running.
3. **Throughput:** Concurrency improves the overall performance of the system by handling more tasks in less time.
4. **Better Use of Multitasking:** Modern operating systems and hardware support multitasking and multi-threading, where concurrency plays a key role.

Principles of Concurrency

Concurrency in operating systems is based on certain principles that ensure it is implemented correctly and efficiently. These principles are critical to avoid conflicts, errors, or inconsistencies during process execution.

1. *Interleaving and Overlapping*

- **Interleaving:** When the operating system rapidly switches between processes, it may seem like they are executing simultaneously. In reality, each process runs for a small amount of time before the OS switches to another.
- **Overlapping:** Processes may perform different tasks simultaneously, such as computation (CPU-bound) and I/O operations (I/O-bound).

2. *Process Synchronization*

- When multiple processes share resources, **synchronization** is required to ensure the correct order of execution and avoid conflicts.

- Synchronization mechanisms like **semaphores**, **mutexes**, and **monitors** are used to coordinate processes.

3. Mutual Exclusion

- **Mutual exclusion** ensures that when one process is accessing a shared resource, other processes are prevented from accessing it until the first process finishes.
- This prevents **race conditions**, where two or more processes try to access and modify a resource simultaneously, leading to incorrect results.
- Mutual exclusion can be implemented using **locks**, **semaphores**, and **critical sections**.

4. Deadlock Prevention and Avoidance

- **Deadlock** occurs when two or more processes are waiting for each other to release resources, leading to a situation where none of the processes can proceed.
- The OS uses various strategies to prevent or avoid deadlocks:
 - **Resource Allocation Graph**
 - **Banker's Algorithm**
 - Ensuring processes request resources in an orderly manner.

5. Race Condition Avoidance

- A **race condition** occurs when the outcome of a process depends on the order of execution of concurrent processes.
- The operating system ensures proper synchronization to avoid race conditions.
- Race conditions are avoided using locks, semaphores, and atomic operations.

6. Process Scheduling

- The operating system uses scheduling algorithms to determine the order in which processes are executed.
- Common scheduling algorithms include:
 - **First Come First Serve (FCFS)**
 - **Round Robin (RR)**
 - **Shortest Job Next (SJN)**
- Scheduling ensures fairness and optimal utilization of system resources.

7. Context Switching

- Context switching occurs when the operating system switches the CPU from one process to another.
- The OS saves the state (context) of the current process and loads the state of the next process.
- This ensures processes can resume from where they left off.

Challenges of Concurrency

1. **Deadlock:** Processes waiting indefinitely for each other to release resources.
2. **Race Conditions:** Unpredictable results when processes access shared resources simultaneously.
3. **Starvation:** Some processes may never get a chance to execute if higher-priority processes keep running.
4. **Complexity:** Managing multiple processes efficiently is complex and requires careful synchronization.

Benefits of Concurrency

1. **Improved CPU Utilization:** CPU remains busy while handling multiple processes.
2. **Increased Throughput:** More processes are completed in a given time.
3. **Responsiveness:** Interactive systems become more responsive to user inputs.
4. **Resource Sharing:** Multiple processes share system resources effectively.

What is Mutual Exclusion?

Mutual Exclusion is a principle in operating systems that ensures **only one process** or thread can access a **shared resource** (like memory, file, or printer) at a time. This prevents conflicts and ensures the consistency of data when multiple processes attempt to use a resource simultaneously.

Why is Mutual Exclusion Important?

In a multitasking environment, multiple processes or threads may try to access shared resources. If there is no control over this access:

1. **Race Conditions:** The outcome becomes unpredictable as processes may read or modify the resource simultaneously.
2. **Data Inconsistency:** The shared resource (e.g., a variable or file) may get corrupted.
3. **Resource Conflicts:** Processes can interfere with each other's execution.

Mutual exclusion ensures that only one process accesses the shared resource at a time, avoiding these issues.

How is Mutual Exclusion Implemented?

To implement mutual exclusion, the following mechanisms are commonly used:

1. **Critical Section:**
2. **Locks (Mutex):**
3. **Semaphores:**
4. **Monitors:**
5. **Atomic Operations:**

What is a Race Condition?

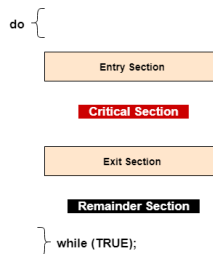
A **race condition** occurs in a multi-process or multi-threaded system when two or more processes or threads access a **shared resource** (like a variable, memory, or file) simultaneously, and the **outcome** depends on the **order of execution**. Race conditions lead to **unpredictable behavior** and can cause incorrect results, data corruption, or system crashes.

What is a Critical Section?

A **Critical Section** is a specific part of a program where a process or thread **accesses or modifies shared resources** such as variables, memory, or files. It is critical because **concurrent access** to these shared resources by multiple processes or threads can lead to issues like **race conditions**, data corruption, or inconsistency.

In simple terms, a critical section is a group of instructions/statements or regions of code that need to be executed atomically, such as accessing a resource (file, input or output port, global data, etc..).

A diagram that demonstrates the critical section is as follows –



Solution to the Critical Section Problem

The critical section problem needs a solution to synchronize the different processes. The solution to the critical section problem must satisfy the following conditions –

1. **Mutual Exclusion:** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress:** Progress means that if no process is inside the critical section, and there are processes waiting to enter it, then the decision about which process enters next must not be delayed unnecessarily. A process that is not using the critical section should not prevent other processes from accessing it.
3. **Bounded Waiting:** Each process must get a chance to enter the critical section within a finite amount of time to avoid starvation.

There are two main approaches to handle critical sections in operating systems:

1. **Preemptive Kernels:** In a preemptive kernel, a process running in **kernel mode** can be **interrupted** (preempted) at any time to allow another process to execute. This makes the system more responsive but requires careful handling to avoid race conditions.
2. **Non-Preemptive Kernels:** In a non-preemptive kernel, once a process starts running in **kernel mode**, it **cannot be interrupted**. The process will continue to run until: It **exits kernel mode**, It **blocks** (e.g., waits for I/O) or It **voluntarily gives up** the CPU.

This approach avoids race conditions on **kernel data structures** because only one process runs in the kernel at a time.

Advantages of Critical Section:

- **Prevents race conditions:** Ensures only one process accesses the critical section at a time, avoiding data inconsistencies.
- **Provides mutual exclusion:** Prevents multiple processes from accessing the same resource simultaneously.
- **Reduces CPU utilization:** Allows processes to wait efficiently without wasting CPU resources.

Disadvantages of Critical Section:

- **Overhead:** Using synchronization mechanisms like semaphores and mutexes can slow down execution due to added overhead.
- **Deadlocks:** Improperly implemented critical sections can cause deadlocks, where processes wait indefinitely for resources.
- **Can limit parallelism:** Large or frequent critical sections can reduce parallelism, affecting overall performance.
- **Can cause contention**

What is Peterson's Solution in Operating Systems?

Peterson's Solution is a classical algorithm in **operating systems** for **mutual exclusion**, designed to solve the **critical section problem** for **two processes** that need to access shared resources. It was introduced by **Gary Peterson** in 1981.

Peterson's Solution is a **software-based** synchronization algorithm used to solve the **critical section problem** in a **two-process** system. It ensures that two processes can safely share a critical section without causing **race conditions**, by providing mutual exclusion, progress, and bounded waiting.

How Peterson's Solution Works

Peterson's solution uses two shared variables to control the access of the two processes to the critical section:

1. boolean flag[2] :

Each process has a corresponding flag that indicates whether it wants to enter the critical section.

- `flag[0] = true` means Process P0 wants to enter the critical section.
- `flag[1] = true` means Process P1 wants to enter the critical section.

2. int turn:

This variable indicates whose turn it is to enter the critical section. It is used to prevent both processes from entering the critical section at the same time.

- If `turn == 0` means it's process 0's turn.
- If `turn == 1` means it's process 1's turn.

```
int turn;
boolean flag[2];
while (true) {
    flag[i] = true;
    turn = i;
    while (flag[j] && turn == j);
    /* critical section */
    flag[i] = false;
    /*remainder section */
}
```

Algorithm (Structure of Process P_i)

Each process follows the same basic structure when trying to enter the critical section:

1. **Set `flag[i] = true`:** This signals that process P_i wants to enter the critical section.

2. **Set turn = j:** This gives priority to the other process (Pj) if both processes want to enter the critical section.
3. **Wait until flag[j] == false or turn == i:** Process Pi will keep checking whether Process Pj is still interested in entering the critical section or if it's Pi's turn.
4. **Enter the critical section:** If Pi passes the above conditions, it enters the critical section to execute its task.
5. **Set flag[i] = false:** After exiting the critical section, Pi marks that it no longer needs the critical section, allowing the other process to enter if needed.

This ensures that at most one process can enter the critical section at any time.

Advantages of Peterson's Solution

- **Simple and Elegant:** The algorithm is easy to understand and implement for two processes.
- **No Hardware Requirements:** Unlike other solutions (e.g., test-and-set), Peterson's solution is purely software-based and doesn't require special hardware support.

Disadvantages of Peterson's Solution

- It involves busy waiting. (In the Peterson's solution, the code statement- "while(flag[j] && turn == j);" is responsible for this. Busy waiting is not favored because it wastes CPU cycles that could be used to perform other tasks.)
- It is limited to 2 processes.
- Peterson's solution cannot be used in modern CPU architectures.
- Potential for starvation in rare or specific cases.

Hardware support for synchronization

There are three algorithms in the hardware approach of solving Process Synchronization problem:

1. Test and Set
2. Swap
3. Unlock and Lock

Test and Set

The **Test and Set** instruction is a hardware-based synchronization mechanism used to implement mutual exclusion. It involves an atomic operation that both tests and sets the value of a memory location.

- The **TestandSet()** operation works with a shared variable, which is typically called `lock`. This variable is initialized to **false** (indicating the critical section is unlocked).
- The **TestandSet(lock)** always **returns** the current value of the `lock` and then **sets** the `lock` variable to **true** (indicating the critical section is now locked).
- The process **first reads** the value of the variable. If the value is **false** (unlocked), it **sets** the variable to **true** (locked) to take control of the critical section.
- If the value of the `lock` is already **true** (meaning the critical section is locked by another process), the process knows it must **wait**. It will keep checking the variable until it becomes **false** again (indicating the critical section is unlocked).

Code:

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}  
  
do {  
    while (test_and_set(&lock));  
    /* do nothing */  
    /* critical section */  
    lock = false;  
    /* remainder section */  
} while (true);
```

Unlock and lock

Unlock and lock algorithm uses the TestAndSet method to control the value of lock. Unlock and lock algorithm uses a variable waiting[i] for each process i. Here i is a positive integer i.e 1,2,3,... which corresponds to processes P1, P2, P3... and so on. waiting[i] checks if the process i is waiting or not to enter into the critical section.

All the processes are maintained in a ready queue before entering into the critical section. The processes are added to the queue with respect to their process number. The queue is the circular queue.

Let's see the Unlock and lock algorithm pseudo-code first:

```
do{  
    Acquire lock;  
    Critical section  
    Release lock  
    Remainder section  
While(true);
```

In Unlock and lock algorithm the lock is not set to false as one process comes out of the critical section. In other algorithms like swap and Test and set the lock was being set to false as the process comes out of the critical section so that any other process can enter the critical section.

But in Unlock and lock, once the ith process comes out of the critical section the algorithm checks the waiting queue for the next process waiting to enter the critical section i.e jth process. If there is a jth process waiting in the ready queue to enter the critical section, the waiting[j] of the jth process is set to false so that the while loop while(waiting[i] && key) becomes false and the jth process enters the critical section.

If no process is waiting in the ready queue to enter the critical section the algorithm then sets the lock to false so that any other process comes and enters the critical section easily.

Since a ready queue is always maintained for the waiting processes, the Unlock and lock algorithm ensures bounded waiting.

Compare and Swap (CAS)

The **Compare and Swap (CAS)** instruction, similar to the **Test and Set** instruction, provides atomic operations to solve mutual exclusion problems in concurrent systems. CAS is widely used for synchronization in modern multi-core systems.

Definition of Compare and Swap

The CAS instruction operates **atomically** on **three operands**:

1. **value**: A pointer to the memory location holding the current value.
2. **expected**: The value we expect to find in the memory location.
3. **new value**: The value to be stored if the comparison succeeds.

How CAS Works

1. The CAS operation checks whether the value at the memory location (***value**) matches the **expected value**.
2. If the values match, the memory location is updated with the **new value**.
3. Regardless of whether the update occurs, the original value of the memory location is returned.

This entire operation is **executed atomically**, meaning that no other operation can interrupt it while it is executing.

CAS

The code for the Compare and Swap instruction is given below:

```
int compare_and_swap(int *value, int expected, int new_value) {  
  
    int temp = *value;          // Store the current value at the memory location  
    if (*value == expected)     // Check if the value equals the expected value  
        *value = new_value;     // Update memory location with the new value  
    return temp;                // Return the original value  
}  
while (true) {  
    while (compare_and_swap(&lock, 0, 1) != 0);  
    /* do nothing */  
    /* critical section */  
    lock = 0;  
    /* remainder section */  
}
```

```
}
```

- **temp** temporarily holds the original value.
- If the current value (*value) matches the expected value, it is replaced with new_value.
- The original value is always returned, indicating success or failure of the swap.

Key Characteristics of CAS

1. **Atomicity:** The CAS operation ensures atomic execution. If two CAS operations execute simultaneously on different cores, they will execute **sequentially** in some arbitrary order.
2. **Mutual Exclusion:** By leveraging CAS, mutual exclusion can be achieved, where only one process can access the critical section at a time.

Mutex Locks

A **mutex lock** (short for *mutual exclusion*) is a simple and widely used synchronization tool to solve the critical-section problem. It ensures that only one process or thread can access a shared resource (or critical section) at a time, thereby preventing race conditions.

The process first **acquires** the lock before entering the critical section and **releases** the lock once it finishes its task. If another process tries to acquire the lock while it is already held, it waits until the lock is released.

To implement a mutex lock, we use a variable that keeps track of whether the lock is free or not. If the lock is available, the process can acquire it by setting the variable to indicate that the lock is now unavailable. Otherwise, the process must wait in a loop (this is called *busy waiting*).

The busy waiting mechanism means the process repeatedly checks for the lock until it becomes available, which wastes CPU time.

Here is a basic implementation of the **acquire()** and **release()** operations for a mutex lock:

```
acquire() {  
    while (!available)  
        ; // Busy waiting  
    available = false; // Lock acquired  
}  
  
release() {  
    available = true; // Lock released  
}
```

Busy Waiting (Spinlocks)

- The acquire() function in its current form causes **busy waiting**, meaning the process keeps checking the lock in a loop until it becomes free.
- While waiting, the process consumes CPU cycles unnecessarily, which is a problem in multiprogramming systems.
- This type of lock is called a **spinlock** because the process "spins" while waiting.

Lock Contention

Lock contention occurs when multiple threads or processes try to acquire the same lock at the same time, causing them to wait for access to the shared resource. If the lock is already held by another thread, other threads either block or spin (busy wait) until the lock becomes available.

There are two types of lock contention:

1. **Uncontended Lock:** No competition for the lock, so execution proceeds without delay.
2. **Contended Lock:** Multiple threads are waiting for the lock, causing delays.

Advantages of Mutex Locks

1. **Simple and Effective:** Ensures mutual exclusion to protect shared resources.
2. **No Context Switch for Spinlocks:** Saves time if the critical section is short.
3. **Commonly Used:** Mutex locks are widely used in operating systems and programming libraries like Pthreads.

Disadvantages of Mutex Locks

1. **Busy Waiting:** In spinlocks, CPU cycles are wasted when waiting for the lock.
2. **Performance Issues:** High contention on locks can slow down applications.
3. **Deadlocks:** Improper usage can lead to deadlocks where no process can proceed.

What is Semaphore?

A semaphore is a non-negative integer variable that is shared between various threads. They are used to enforce mutual exclusion, avoid race conditions, and implement synchronization between processes.

Semaphore is accessed only through two standard atomic operations: `wait()` and `signal()`.

Semaphores were introduced by the Dutch computer scientist Edsger Dijkstra,

The wait operation decrements the value of the semaphore, and the signal operation increments the value of the semaphore.

Wait operation: When a process tries to use a resource, it checks the semaphore's value:

- If the value is greater than 0, the process can continue and the semaphore value decreases by 1 (indicating the resource is now being used).
- If the value is 0, the process is blocked (it has to wait until the semaphore value becomes greater than 0).

The definition of wait() is as follows:

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

Signal operation: When a process finishes using a resource, it signals by increasing the semaphore value by 1. This can either:

- Wake up a process that was waiting for the resource (if any), or
- Allow other processes to continue if there is no one waiting.

The definition of signal() is as follows:

```
signal(S) {  
    S++;  
}
```

Semaphores are also called counting semaphores because the value of the semaphore counts how many processes can access the resource. The initial value of the semaphore determines how many processes can use the resource at once.

Types of Semaphores:

1. **Counting Semaphore:** This type of semaphore can have any integer value, allowing it to represent a pool of available resources. For example, if there are multiple

instances of a resource (like a printer), a counting semaphore can track how many are free.

2. **Binary Semaphore:** This type can only have two values, 0 or 1. It behaves similarly to a mutex lock and is used to ensure mutual exclusion between processes.

Semaphore Implementation:

In semaphore implementation, instead of busy waiting (where a process continuously checks if a resource is available), a process **suspends** itself if the semaphore value is not positive. This means the process is put to sleep and added to a waiting queue. The CPU scheduler then selects another process to run.

When a process performs a `signal()` operation, if any processes are waiting, one is **woken up** and moved to the ready state. It will then be scheduled to run by the CPU. This avoids wasting CPU time and makes the system more efficient.

To implement semaphores under this definition, we define a semaphore as follows:

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Each semaphore has an integer value and a list of processes list. When a process must wait on a semaphore, it is added to the list of processes. A `signal()` operation removes one process from the list of waiting processes and awakens that process.

Now, the `wait()` semaphore operation can be defined as

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        sleep();  
    }  
}
```

and the `signal()` semaphore operation can be defined as

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

Advantages of Semaphores

- A simple and effective mechanism for process synchronization
- Supports coordination between multiple processes
- Provides a flexible and robust way to manage shared resources.
- It can be used to implement critical sections in a program.
- It can be used to avoid race conditions.

Disadvantages of Semaphores

- It Can lead to performance degradation due to overhead associated with wait and signal operations.
- Can result in deadlock if used incorrectly.
- It can cause performance issues in a program if not used properly.
- It can be difficult to debug and maintain.
- It can be prone to race conditions and other synchronization problems if not used correctly.

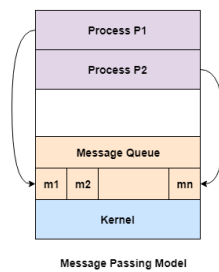
Message Passing Model of Process Communication

Process communication is the mechanism provided by the operating system that allows processes to communicate with each other. This communication means one process informing another that something has happened or sending data from one process to another.

One of the models of process communication is the message passing model.

Message passing model allows multiple processes to read and write data to the message queue without being connected to each other. Messages are stored on the queue until their recipient retrieves them. Message queues are quite useful for interprocess communication and are used by most operating systems.

A diagram that demonstrates message passing model of process communication is given as follows –



In the above diagram, both the processes P1 and P2 can access the message queue and store and retrieve data.

Advantages of Message Passing Model

Some of the advantages of message passing model are given as follows –

- The message passing model is much easier to implement than the shared memory model.
- It is easier to build parallel hardware using message passing model as it is quite tolerant of higher communication latencies.

Disadvantage of Message Passing Model

- The message passing model has slower communication than the shared memory model because the connection setup takes time.

What Is a Signal?

A signal is an asynchronous notification sent to a process or thread in an operating system to indicate a specific event or condition. These signals can be generated by the operating system itself, by other processes, or even by the process itself.

Why Do Signals Exist?

Signals exist to facilitate communication and coordination between processes. They provide an effective way to handle events and conditions in an operating system. Some common examples of signals include process termination (SIGTERM), process interruption (SIGINT), or detection of an error (SIGSEGV).

When Are Signals Delivered and by Whom?

Signals can be delivered at various times:

By the Operating System: The operating system can send signals to processes to inform them about system events, such as the termination of a child process or a request for interruption.

By Other Processes: Processes can also send signals to other processes, allowing communication between them. This is useful for coordinating activities or notifying events

What is a Monitor in OS?

A **monitor** is a synchronization tool that provides a high-level abstraction for managing concurrent processes. It is used to avoid race conditions and ensure mutual exclusion, making it easier for programmers to safely manage shared resources without worrying about low-level details like locking.

The Monitor is a package that contains shared data structures, operations, and synchronization between concurrent procedure calls. Therefore, a monitor is also known as a synchronization tool.

Java, C#, Visual Basic, Ada, and concurrent Euclid are among some of the languages that allow the use of monitors.

Processes operating outside the monitor can't access the monitor's internal variables, but they can call the monitor's procedures.

Syntax of Monitor in OS

```
Monitor monitorName{
    variables_declaration;
    condition_variables;

    procedure p1{ ... };
    procedure p2{ ... };
    ...
    procedure pn{ ... };

    {
        initializing_code;
    }}
}
```

Monitor in an operating system is simply a class containing variable_declarations, condition_variables, various procedures (functions), and an initializing_code block that is used for process synchronization.

Components of Monitor in an Operating System

The monitor is made up of four primary parts:

1. **Initialization:** The code for initialization is included in the package, and we just need it once when creating the monitors.
2. **Private Data:** It is a feature of the monitor in an operating system to make the data private. It holds all of the monitor's secret data, which includes private functions that may only be utilized within the monitor. As a result, private fields and functions are not visible outside of the monitor.
3. **Monitor Procedure:** Procedures or functions that can be invoked from outside of the monitor are known as monitor procedures.
4. **Monitor Entry Queue:** It contains all of the threads, which are commonly referred to as procedures only.

Condition Variables

There are two sorts of operations we can perform on the monitor's condition variables:

1. Wait

2. Signal

Consider a condition variable (y) is declared in the monitor:

`y.wait()`: The activity/process that applies the wait operation on a condition variable will be suspended, and the suspended process is located in the condition variable's block queue.

`y.signal()`: If an activity/process applies the signal action on the condition variable, then one of the blocked activity/processes in the monitor is given a chance to execute.

Advantages of Monitor in OS

- Monitors offer the benefit of making concurrent or parallel programming easier and less error-prone than semaphore-based solutions.
- It helps in process synchronization in the operating system.
- Monitors have built-in mutual exclusion.
- Monitors are easier to set up than semaphores.
- Monitors may be able to correct for the timing faults that semaphores cause.

Disadvantages of Monitor in OS

- Monitors must be implemented with the programming language.
- Monitor increases the compiler's workload.
- The monitor requires to understand what operating system features are available for controlling crucial sections in the parallel procedures.

Classical problems of synchronization

Bounded Buffer problem

Bounded buffer problem, which is also called producer consumer problem, is one of the classic problems of synchronization. involves a producer that generates data and a consumer that processes the data.

The data is stored in a shared buffer with a limited capacity. The buffer is of n slots and each slot is capable of storing one unit of data.

There are two processes running namely, producer and consumer, which are operating on this buffer.

The producer tries to insert data into an empty slot of the buffer.

The consumer tries to remove data from a filled slot in the buffer

The challenge is to make sure:

- The producer doesn't add items when the buffer is full.
- The consumer doesn't try to take items when the buffer is empty.
- The producer and consumer should not insert and remove data simultaneously.

We will make use of three semaphores:

1. **m(mutex)**, a binary semaphore which is used to acquire and release the lock
2. **empty**, a counting semaphore whose initial value is the number of slots in the buffer, since, initially all slots are empty.
3. **full**, a counting semaphore whose initial value is 0.

Producer code:

```
do{
wait(empty);      //wait until empty>0 and then decrement empty
Wait(mutex);     //acquire lock
/*add data to buffer*/
Signal(mutex);    //release lock
Signal(full);     //increment full
}while(TRUE);
```

Consumer code:

```
do{
    Wait(full);    //wait untill full>0 and then decrement full
    Wait(mutex);  acquire lock
    /*remove data from buffer*/
    Signal(mutex); //release lock
    Signal(empty); //increment empty
}while(TRUE);
```

The Readers-Writers Problem?

The Readers-Writers Problem is a classic synchronization issue in operating systems that involves managing access to shared data by multiple threads or processes. The problem addresses the scenario where:

- **Readers:** Multiple readers can access the shared data simultaneously without causing any issues because they are only reading and not modifying the data.
- **Writers:** Only one writer can access the shared data at a time to ensure data integrity, as writers modify the data, and concurrent modifications could lead to data corruption or inconsistencies.

Challenges of the Reader-Writer Problem

The challenge now becomes how to create a synchronization scheme such that the following is supported:

- **Multiple Readers:** A number of readers may access simultaneously if no writer is presently writing.
- **Exclusion for Writers:** If one writer is writing, no other reader or writer may access the common resource.

Solution using Semaphore:

Code for Writer Process:

```
while (true) {
    wait(rw_mutex);
    /* writing is performed */
    signal(rw_mutex);
}
```

Code for Reader Process:

```
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
```

```
    signal(mutex);

    /* reading is performed */

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
}
```

The Dining Philosophers Problem

The dining philosopher's problem is the classical problem of synchronization which says that Five philosophers are sitting around a circular table and their job is to think and eat alternatively.

A bowl of noodles is placed at the center of the table along with five chopsticks for each of the philosophers.

To eat a philosopher needs both their right and a left chopstick. A philosopher can only eat if both immediate left and right chopsticks of the philosopher is available. In case if both immediate left and right chopsticks of the philosopher are not available then the philosopher puts down their (either left or right) chopstick and starts thinking again.

Semaphore Solution:

One way to solve this problem is by using semaphores to represent the chopsticks. Each chopstick is represented as a semaphore initialized to 1 (available).

```
semaphore chopstick[5]; // Each chopstick is represented as a semaphore.
```

Structure of Philosopher Process:

```
while (true) {
    wait(chopstick[i]);           // Pick up left chopstick.
    wait(chopstick[(i + 1) % 5]); // Pick up right chopstick.

    /* Eat for a while */
}
```

```
    signal(chopstick[i]);           // Put down left chopstick.  
    signal(chopstick[(i + 1) % 5]); // Put down right chopstick.  
  
    /* Think for a while */  
}
```

Issues: While this solution ensures mutual exclusion (no two philosophers can use the same chopstick simultaneously), it can lead to a **deadlock** if all philosophers pick up their left chopstick at the same time.

Deadlock Prevention:

To avoid deadlock, the following modifications can be made:

1. **At Most Four Philosophers:** Allow a maximum of four philosophers to sit at the table simultaneously.
2. **Critical Section for Both Chopsticks:** A philosopher picks up both chopsticks only if both are available. If either is unavailable, they wait.
3. **Asymmetric Solution:** Use an odd-even approach:
 - a. Odd-numbered philosophers pick up the left chopstick first, then the right.
 - b. Even-numbered philosophers pick up the right chopstick first, then the left.

Monitor Solution (Advanced Deadlock-Free Approach):

A more sophisticated solution uses **monitors** to ensure no deadlocks. A monitor is a high-level synchronization construct that uses condition variables.

```
monitor DiningPhilosophers {  
    enum {THINKING, HUNGRY, EATING} state[5];  
    condition self[5];  
    void pickup(int i) {  
        state[i] = HUNGRY;  
        test(i);  
        if (state[i] != EATING)
```



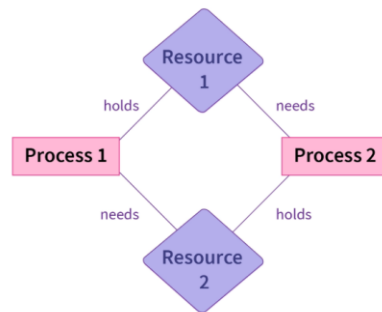
```
        self[i].wait();
    }
void putdown(int i) {
    state[i] = THINKING;
    test((i + 4) % 5);
    test((i + 1) % 5);
}
void test(int i) {
    if ((state[(i + 4) % 5] != EATING) && (state[i] == HUNGRY) && (state[(i + 1) % 5] != EATING)) {
        state[i] = EATING;
        self[i].signal();
    }
}
Initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
}
```

What is Deadlock?

A deadlock is a situation where a set of processes is blocked because each process is holding a resource and waiting for another resource acquired by some other process.

A situation occurs in operating systems when there are two or more processes that hold some resources and wait for resources held by other(s). For example, in the below diagram, Process 1 is holding Resource 1 and

waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.



Necessary Conditions for Deadlock in OS

The four necessary conditions for a deadlock to arise are as follows.

1. **Mutual Exclusion:** Only one process can use a resource at any given time i.e. the resources are non-sharable.
2. **Hold and wait:** A process is holding at least one resource at a time and is waiting to acquire other resources held by some other process.
3. **No Preemption:** A resource cannot be taken from a process unless the process releases the resource.
4. **Circular Wait:** A set of processes are waiting for each other in a circular fashion. For example, let's say there are a set of processes {P0 , P1, P2, P3} such that P0 depends on P1 , P1 depends on P2, P2 depends on P3 and P3 depends on P0 . This creates a circular relation between all these processes and they have to wait forever to be executed.

These conditions are collectively known as the **Coffman conditions**.

Resource Allocation Graph:

Deadlock can be described more clearly by directed graph which is called system resource allocation graph.

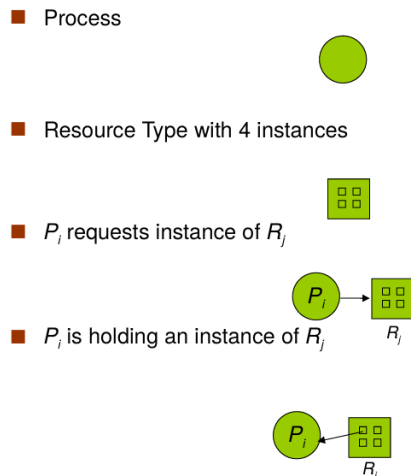
The graph consists of a set of vertices 'V' and a set of edges 'E'.

The set of vertices 'V' is partitioned into two different types of nodes such as $P = \{P1 , P2 , \dots, Pn \}$, the set of all the active processes in the system and $R = \{R1 , R2 , \dots, Rm \}$, the set of all the resource type in the system.

A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$. It signifies that process P_i is an instance of resource type R_j and waits for that resource.

A directed edge from resource type R_j to the process P_i which signifies that an instance of resource type R_j has been allocated to process P_i .

A directed edge $P_i \rightarrow R_j$ is called as **request edge** and $R_j \rightarrow P_i$ is called as **assigned edge**.



When a process P_i requests an instance of resource type R_j then a request edge is inserted as resource allocation graph. When this request can be fulfilled, the request edge is transformed to an assignment edge. When the process no longer needs access to the resource it releases the resource and as a result the assignment edge is deleted.

The resource allocation graph shown in below figure has the following situation.

- The sets P, R, E

$P = \{P_1, P_2, P_3\}$

$R = \{R_1, R_2, R_3, R_4\}$

$E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$

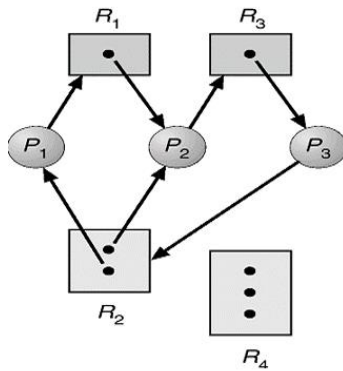
The resource instances are:

- Resource R_1 has one instance
- Resource R_2 has two instances.
- Resource R_3 has one instance
- Resource R_4 has three instances.

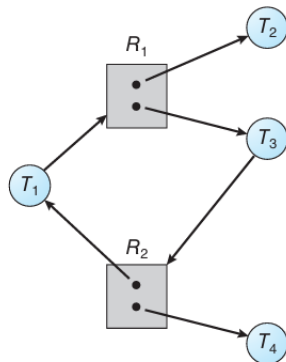
The process states are:

- Process P1 is holding an instance of R2 and waiting for an instance of R1 .
- Process P2 is holding an instance of R1 and R2 and waiting for an instance R3 .
- Process P3 is holding an instance of R3 .

The following example shows the resource allocation graph with a deadlock.



Example 2:



Resource-allocation graph with a cycle but no deadlock.

Methods for Handling Deadlocks

The problem of deadlock can deal with the following 3 ways:

1. We can use a protocol to prevent or avoid deadlock ensuring that the system will never enter to a deadlock state.
2. We can allow the system to enter a deadlock state, detect it and recover.
3. We can ignore the problem all together.

To ensure that deadlock never occur the system can use either a deadlock prevention or deadlock avoidance scheme.

Deadlock Prevention

Deadlock prevention is a method to ensure that at least one of the four necessary conditions for deadlock cannot hold. By addressing each condition, we can eliminate the possibility of deadlock in a system.

1. Mutual Exclusion

Mutual exclusion means that some resources cannot be shared between multiple processes at the same time. For example, resources like printers can only be used by one thread at a time. On the other hand, resources like read-only files can be shared and are not likely to cause deadlocks.

Since some resources are naturally non-sharable, it is not always possible to prevent deadlocks by eliminating mutual exclusion.

2. Hold and Wait

This condition occurs when a process holds one resource and waits for additional resources. To prevent this, the system can follow one of these protocols:

1. A process must request all required resources before starting execution.
2. A process can only request resources when it is holding none.

However, these protocols have drawbacks. They lead to poor resource utilization, as some resources may remain unused for extended periods. Additionally, they can cause starvation, where a process might wait indefinitely for resources that are continuously allocated to others.

3. No Preemption

To address this condition, a protocol can be used where:

- If a process holding resources requests another resource that is unavailable, all its currently held resources are preempted and added to the waiting list.
- The process is restarted only when it can acquire all the resources it needs, including those that were preempted.

This approach works well for resources that can be easily saved and restored, such as CPU registers. However, it is not suitable for resources like mutex locks, where saving and restoring the state is not feasible.

4. Circular Wait

This condition can be violated by not allowing the processes to wait for resources in a cyclic manner.

To violate this condition, the following approach is followed-

Approach-

- A natural number is assigned to every resource.
- Each process is allowed to request for the resources either in only increasing or only decreasing order of the resource number.
- In case increasing order is followed, if a process requires a lesser number resource, then it must release all the resources having larger number and vice versa.
- This approach is the most practical approach and implementable.
- However, this approach may cause starvation but will never lead to deadlock.

Deadlock Avoidance

Deadlock avoidance is a way for operating systems to stop deadlocks from happening by carefully handling resource requests. Unlike deadlock prevention, which uses strict rules to stop deadlocks completely, deadlock avoidance allows processes to request resources but checks each request to make sure it won't cause a problem.

Safe state:

When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state.

Systems are in safe state if there exists a safe sequence of all process. A sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes is the system such that for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$. That is:

- If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished.

- When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate.
- When P_i terminates, P_{i+1} can obtain its needed resources, and so on.
- If system is in safe state $\Rightarrow$ No deadlock
- If system is not in safe state $\Rightarrow$ possibility of deadlock
- OS cannot prevent processes from requesting resources in a sequence that leads to deadlock
- Avoidance $\Rightarrow$ ensure that system will never enter an unsafe state, prevent getting into deadlock

Example:

	Maximum Needs	Current Needs
P_0	10	5
P_1	4	2
P_2	9	2

- Suppose processes P_0 , P_1 , and P_2 share 12 magnetic tape drives
- Currently 9 drives are held among the processes and 3 are available
- Question: Is this system currently in a safe state?
- Answer: Yes!
 - Safe Sequence: $\langle P_1, P_0, P_2 \rangle$

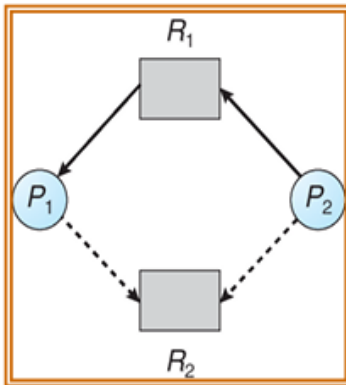
	Maximum Needs	Current Needs
P_0	10	5
P_1	4	2
P_2	9	2

- Suppose process P_2 requests and is allocated 1 more tape drive.
- Question: Is the resulting state still safe?
- Answer: No! Because there does not exist a safe sequence anymore.
 - Only P_1 can be allocated its maximum needs.
 - If P_0 and P_2 request 5 more drives and 6 more drives, respectively, then the resulting state will be deadlocked.

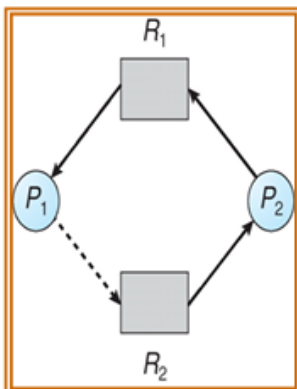
Resource Allocation Graph Algorithm

In this graph a new type of edge has been introduced is known as **claim edge**.

Claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j ; represented by a dashed line. Claim edge converts to request edge when a process requests a resource. Request edge converted to an assignment edge when the resource is allocated to the process. When a resource is released by a process, assignment edge reconverts to a claim edge. Resources must be claimed a priori in the system.



- P_2 requesting R_1 , but R_1 is already allocated to P_1 .
- Both processes have a claim on resource R_2
- What happens if P_2 now requests resource R_2 ?



- Cannot allocate resource R_2 to process P_2
- Why? Because resulting state is unsafe
 - P_1 could request R_2 , thereby creating deadlock!